

*¿Qué son los estados en React? ¿Cuáles son sus características y para qué sirve?
Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo
descripto, explicando el ejemplo.*

El estado es un objeto interno de un componente que almacena datos que pueden cambiar a lo largo del tiempo y que afectan lo que se renderiza en pantalla. A diferencia de una variable común, cuando el estado cambia, React vuelve a renderizar automáticamente el componente para reflejar ese cambio.

Características principales:

- Es local al componente (a menos que se comparta explícitamente vía props o context).
- Es mutable, pero solo debe modificarse a través de funciones especiales (como setState o el setter que devuelve useState), nunca directamente.
- Cada cambio de estado dispara un re-render del componente.
- Es asíncrono en su actualización: React puede agrupar varias actualizaciones antes de re-renderizar.
- Sirve para manejar datos dinámicos: formularios, contadores, respuestas de APIs, toggles de UI, etc.

```
import { useState } from 'react';

function Contador() {
  const [contador, setContador] = useState(0);

  return (
    <div>
      <p>Valor actual: {contador}</p>
      <button onClick={() => setContador(contador + 1)}>Sumar</button>
      <button onClick={() => setContador(contador - 1)}>Restar</button>
    </div>
  );
}

export default Contador;
```

*¿Qué son los eventos en React? ¿Cuáles son sus características y para qué sirve?
Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo
descripto, explicando el ejemplo.*

Los eventos en React son las acciones del usuario que el componente puede “escuchar” y responder mediante funciones (event handlers).

Características principales:

- Se manejan de forma declarativa dentro del JSX, con atributos tipo onClick, onChange, onSubmit, etc. (en camelCase, a diferencia del HTML plano).
- React usa un sistema de eventos sintéticos (SyntheticEvent), una envoltura sobre los eventos nativos del DOM que unifica el comportamiento entre navegadores.
- El handler recibe el evento como parámetro, del cual se puede leer información (event.target.value, por ejemplo).
- Permiten prevenir comportamientos por defecto con event.preventDefault().
- Sirven para conectar la interacción del usuario con cambios de estado o lógica del componente.

```
import { useState } from 'react';

function FormularioNombre() {
  const [nombre, setNombre] = useState('');

  const manejarCambio = (event) => {
    setNombre(event.target.value);
  };

  const manejarEnvio = (event) => {
    event.preventDefault();
    alert(`Hola, ${nombre}!`);
  };

  return (
    <form onSubmit={manejarEnvio}>
      <input type="text" value={nombre} onChange={manejarCambio} placeholder="Escribí tu nombre" />
      <button type="submit">Saludar</button>
    </form>
  );
}

export default FormularioNombre;
```

*¿Qué son los Hook en React? ¿Cuáles son sus características y para qué sirve?
 ¿Cuáles son los principales Hook? Desarrollar un ejemplo sencillo en código
 REACT que permita visualizar lo descripto, explicando el ejemplo.*

Los Hooks son funciones especiales que permiten usar estado, efectos y otras capacidades de React dentro de componentes funcionales. Antes de que aparecieran las capacidades se asociaban a componentes de clase, con los hooks la opción principal pasó a ser el modelo principal.

Algunos hooks fundamentales son:

- useState: para estado local.
- useEffect: para sincronizar efectos secundarios.
- useMemo y useCallback: para optimización de cálculos y referencias.
- useRef: para referencias mutables o acceso al DOM.

```
import { useState, useEffect } from 'react';

function RelojDigital() {
  const [hora, setHora] = useState(new Date());

  useEffect(() => {
    const intervalo = setInterval(() => {
      setHora(new Date());
    }, 1000);

    // Limpieza: se ejecuta cuando el componente se desmonta
    return () => clearInterval(intervalo);
  }, []); // Array vacío: el efecto corre una sola vez, al montar

  return <h2>Hora actual: {hora.toLocaleTimeString()}</h2>;
}

export default RelojDigital;
```